



Synthèse complète : Éléments fondamentaux de la programmation Java



1. Introduction et vocabulaire de base

- **Java vs C++** : Bien que proches, Java est plus sécurisé et simplifié (pas de gestion manuelle de la mémoire).
- **Caractères** : Utilise le **jeu UNICODE** (16 bits), permettant la gestion de milliers de symboles.
- **Portabilité** : Grâce au **bytecode** exécuté sur la **JVM**, un programme Java est indépendant de la plateforme.



2. Variables et Types en Java

➤ Types de variables :

1. **Primitifs** : Types de base (int, float, char, boolean, etc.).
2. **Références** : Pointeurs vers des objets ou tableaux.
 - Déclaration d'un objet : `Date birthday;` (Ne crée pas l'objet).
 - Allocation avec `new` : `birthday = new Date(74, 1, 7);` .

➤ Tableaux :

- Déclaration : `int[] tab = new int[3];`
- Initialisation : `int[] tab = {23, 3, 79};`
- Tableaux d'objets :

```
Date[] dates = new Date[4];  
dates[0] = new Date(58, 0, 23);
```



3. Constantes et Opérateurs

- **Constantes** : Utilisation du mot-clé `final` .

Exemple :

```
final int EXCHANGE = 1;
```

- **Opérateur** `instanceof` : Teste si un objet est une instance d'une classe :

```
if (obj instanceof Date) { ... }
```

4. Classes et Objets en Java

- Toute classe hérite par défaut de la classe **Object**.

Méthodes importantes d'

`Object` :

- `equals()` : Compare le contenu des objets.
- `toString()` : Fournit une représentation textuelle.
- `clone()` : Crée une copie d'un objet (nécessite `Cloneable`).

➤ Exemple de classe :

```
public class Personne {
    private String nom;
    private int age;
    private double salaire;

    public Personne(String nom, int age, double salaire) {
        this.nom = nom;
        this.age = age;
        this.salaire = salaire;
    }

    @Override
    public String toString() {
        return "Personne{" + "nom=" + nom + ", age=" + age + ", salaire=" + salaire + "}";
    }
}
```

5. Héritage

- Mot-clé : `extends` pour hériter d'une autre classe.
- Utilisation de `super()` pour appeler le constructeur parent.
- Java **n'autorise pas l'héritage multiple**.

Exemple :

```
public class Employe extends Personne {  
    private int matricule;  
    public Employe(String nom, int age, int matricule) {  
        super(nom, age);  
        this.matricule = matricule;  
    }  
}
```

6. Les Packages

- Organisation de classes liées dans un même espace :
 - Déclaration : `package monPackage;`
 - Importation : `import monPackage.MaClasse;`
- Structure : Chaque package correspond à un **répertoire**.

Exemple :

```
package hepl.java.basics;
```

7. Les Interfaces

- Définition : Un contrat que les classes doivent respecter (similaire aux classes abstraites).
- Déclaration :

```
public interface Identifiable {  
    String getId();  
}
```

- Implémentation :

```
public class Employe implements Identifiable {
    public String getId() {
        return "12345";
    }
}
```

- Une classe peut **implémenter plusieurs interfaces**, mais **pas hériter de plusieurs classes**.

! 8. Exceptions en Java

- **Définition** : Une erreur pendant l'exécution du programme.

- **Gestion** :

```
try {
    int x = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Erreur : " + e.getMessage());
} finally {
    System.out.println("Toujours exécuté");
}
```

- **Créer sa propre exception** :

```
public class MonException extends Exception {
    public MonException(String message) {
        super(message);
    }
}
```

AB CD 9. Gestion des chaînes de caractères

- **String** : Immutable (ne peut pas être modifiée).
- **StringBuffer** : Modifiable (meilleures performances pour les modifications répétées).

➤ Méthodes importantes :

```
String nom = "Java";  
int longueur = nom.length();  
StringBuffer sb = new StringBuffer("Bonjour");  
sb.append(" le monde");
```

10. Clonage des objets

- Pour **cloner** un objet, il doit implémenter l'interface **Cloneable**.
Exemple :

```
public class Personne implements Cloneable {  
    public Object clone() {  
        return new Personne(nom, age, salaire);  
    }  
}
```

11. Types énumérés (Enums)

- Définition : Liste de **valeurs constantes prédéfinies**.
- Exemple :

```
public enum Jour {  
    LUNDI, MARDI, MERCREDI  
}
```

- Utilisation :

```
Jour aujourd'hui = Jour.LUNDI;  
if (aujourd'hui == Jour.LUNDI) { ... }
```

12. Entrées et Sorties

- **Sorties** : `System.out.println("Hello");`
- **Entrées** : Utilisation de la classe **Scanner** :

```
Scanner sc = new Scanner(System.in);  
String input = sc.nextLine();
```



13. Manipulation des dates

- **LocalDate** : Pour manipuler les dates.
- **Exemples :**

```
LocalDate today = LocalDate.now();  
LocalDate specificDate = LocalDate.of(2023, 5, 14);
```

- **Calcul de différences :**

```
Period difference = Period.between(date1, date2);
```



14. Les classes "Wrapper"

- Permettent de manipuler les types primitifs comme des objets (**Integer** , **Double** , etc.).
- Exemple :

```
int x = Integer.parseInt("123");  
String s = Integer.toString(456);
```

✓ Conclusion :

Java est un langage

orienté objet, **portable**, et **sécurisé**, adapté pour des applications allant des programmes **basiques** aux **systèmes complexes**. Ses concepts fondamentaux incluent :

- Les **classes** et **objets**.
- L'**héritage** et les **interfaces**.
- La **gestion des exceptions**.
- La manipulation **des chaînes, des dates et des fichiers**.

Java est essentiel pour le développement d'applications robustes et multiplateformes.

15. Exemples de code avec leurs résultats



Classes et Objets

```
// Définition et utilisation
public class Voiture {
    private String marque;
    private int annee;

    public Voiture(String marque, int annee) {
        this.marque = marque;
        this.annee = annee;
    }

    @Override
    public String toString() {
        return "Voiture: " + marque + " " + annee;
    }

    public static void main(String[] args) {
        Voiture maVoiture = new Voiture("Toyota", 2024);
        System.out.println(maVoiture);
    }
}
```

Résultat :
Voiture: Toyota (2024)



Héritage

```
class Animal {
    protected String nom;

    public void faireBruit() {
        System.out.println("Bruit");
    }
}

class Chat extends Animal {
    @Override
    public void faireBruit() {
        System.out.println("Miaou");
    }
}

public class TestAnimal {
    public static void main(String[] args) {
        Animal chat = new Chat();
        chat.faireBruit();
    }
}
```

Résultat :
Miaou!



Interfaces

```
interface Volant {  
    void voler();  
}  
  
class Oiseau implements Volant {  
    @Override  
    public void voler() {  
        System.out.println("L'oiseau  
    }  
}  
  
public class TestInterface {  
    public static void main(String[]  
        Oiseau moineau = new Oisea  
        moineau.voler();  
    }  
}
```

Résultat :
L'oiseau vole dans le ciel



Gestion des Exceptions

```
public class GestionErreur {  
    public static void main(Strin  
        diviser(10, 0);  
        diviser(10, 2);  
    }  
  
    public static void diviser(int  
        try {  
            int resultat = a / b;  
            System.out.println("Ré  
        } catch (ArithmeticExcept  
            System.out.println("Div  
        } finally {  
            System.out.println("Op  
        }  
    }  
}
```

Résultat :
Division par zéro impossible!
Opération terminée
Résultat: 5
Opération terminée



Manipulation des Chaînes

```
public class ManipulationTexte {  
    public static void main(String[] args) {  
        String texte = "Hello Java";  
        System.out.println("En majuscules: " + texte.toUpperCase());  
  
        StringBuffer sb = new StringBuffer();  
        sb.append(" World");  
        System.out.println("StringBuffer: " + sb.toString());  
    }  
}
```

Résultat :

En majuscules: HELLO JAVA

StringBuffer: Hello World



Manipulation des Dates

```
import java.time.LocalDate;  
import java.time.Period;  
  
public class ManipulationDate {  
    public static void main(String[] args) {  
        LocalDate aujourd'hui = LocalDate.now();  
        LocalDate noel = LocalDate.of(2023, 12, 25);  
  
        Period difference = Period.between(aujourd'hui, noel);  
        System.out.println("Période jusqu'à Noël: " +  
            difference.getMonths() + " mois et " +  
            difference.getDays() + " jours");  
    }  
}
```

Résultat :

Période jusqu'à Noël: 9 mois et 15 jours



Types Énumérés

```

public class TestEnum {
    public enum Saison {
        PRINTEMPS, ETE, AUTOMNE, HIVER
    }

    public static void main(String[] args) {
        Saison saison = Saison.PRINTEMPS;
        switch(saison) {
            case PRINTEMPS:
                System.out.println("C'est le printemps!");
                break;
            case ETE:
                System.out.println("C'est l'été!");
                break;
            default:
                System.out.println("Automne ou Hiver");
        }

        System.out.println("\nToutes les saisons:");
        for(Saison s : Saison.values()) {
            System.out.println(s);
        }
    }
}

```

Résultat :

C'est le printemps!

Toutes les saisons:

PRINTEMPS

ETE

AUTOMNE

HIVER



Classes Wrapper

```

public class TestWrapper {
    public static void main(String[] args) {
        // Conversion String vers int
        String nombreTexte = "123";
        int nombre = Integer.parseInt(nombreTexte);
        System.out.println("String vers int: " + nombre + " (type: " + Integer.class.getName() + ")");

        // Conversion int vers String
        int x = 456;
        String texte = Integer.toString(x);
        System.out.println("Int vers String: " + texte + " (type: " + String.class.getName() + ")");

        // Auto-boxing
        Integer nombreObject = 10;
        System.out.println("Auto-boxing: " + nombreObject + " (type: " + Integer.class.getName() + ")");
    }
}

```

Résultat :

String vers int: 123 (type: Integer)

Int vers String: 456 (type: String)

Auto-boxing: 10 (type: Integer)